

Name: Mohammed Feras Sawab

ID: 2240009066

Lab2: Text Pre-processing and Regular Expressions

In this lab, you will learn essential techniques for preparing and cleaning text data using various pre-processing steps. Additionally, you will gain hands-on experience with regular expressions, a powerful tool for pattern matching and text manipulation in natural language processing (NLP) tasks.

Regular Expressions

Regular expressions (regex) are extremely useful in extracting information from any text by searching for one or more matches of a specific search pattern

<https://www.regexpal.com/>

Regular Expressions use cases

- Data pre-processing
- Pattern matching
- Text feature Engineering
- Web scraping
- Data validation
- Data extraction

An example use case is extracting all hashtags from a tweet, or getting email addresses or phone numbers from large unstructured text content.

Regular Expressions with Python

Python provides a convenient built-in module for managing regular expressions:

```
import re
```

We can import it as we import any other Python library.

Important symbols in Regular Expression

Let's start with the basic regular expression characters and some examples:

- (^) : Matches the expression to its right, at the start of a string before it finds a line break.
 - (\$) : Matches the expression to its left, at the end of a string before it finds a line break.
 - (.) : Matches any character except newline.
 - (a) : Matches exactly one character.
 - (ab) : Matches the string ab.
-

Quantifiers:

- (a|b) : Matches expression a or b. If a is matched first, b is not checked;
 - (+) : Matches the expression to its left 1 or more times;
 - (*) : Matches the expression to its left 0 or more times;
 - (?) : Matches the expression to its left 0 or 1 times.
-

Character Classes:

- \w : Matches alphanumeric characters, that is a-z, A-Z, 0-9, and underscore(_);
 - \W : Matches non-alphanumeric characters, that is except a-z, A-Z, 0-9, and _;
 - \d : Matches digits, from 0-9;
 - \D : Matches any non-digits;
 - \s : Matches whitespace characters, which also include the \t, \n, \r, and space characters;
 - \S : Matches non-whitespace characters.
 - \n : Matches a newline character;
 - \t : Matches a tab character;
 - \b : Matches the word boundary (or empty string) at the start and end of a word;
 - \B : Matches where \b does not, that is non-word boundary.
-

Sets:

- [abc] : Matches either a, b, or c. It does not match abc;
- [a-z] : Matches any alphabet from a to z;
- [A-Z] : Matches any alphabets in capital from A to Z;
- [a-p] : Matches a, -, or p. It matches - because \ escapes it;
- [-z] : Matches - or z;
- [a-z0-9] : Matches characters from a to z or from 0 to 9.
- [(+*)] : Special characters become literal inside a set, so this matches (, +, *, or);
- [^ab5] : Adding ^ excludes any character in the set. Here, it matches characters that are not a, b, or 5;

Regular Expression functions

search

Search for pattern occurrences in a string using the search function of the **re module**. This function returns a match object, containing the matched substring (or None, if it doesn't exist) and its position inside the original string.

In [1]: `import re`

```
text = 'I am enjoying the NLP course.'

print(re.search(r"I", text)) #r is maning i will start regular
print(re.search(r"se.$", text)) #$ look form the end
print(re.search(r"am", text))
print(re.search(r"m", text))
print(re.search(r"AI", text))
```

```
<re.Match object; span=(0, 1), match='I'>
<re.Match object; span=(26, 29), match='se.'>
<re.Match object; span=(2, 4), match='am'>
<re.Match object; span=(3, 4), match='m'>
None
```

In [2]: `match = re.search((r"I"),text)`
`print(match.span())`
`print(match.start())`
`print(match.end())`
`print("The matched pattern is: ",text[match.start():match.end()])` *#index fro*

```
(0, 1)
```

```
0
```

```
1
```

```
The matched pattern is: I
```

match

The match function is similar to search, but it only tries to match the pattern at the beginning of the target string.

In [3]: `import re`

```
text = 'I am enjoying the NLP course.'

print(re.search(r"I.*", text)) #global
print(re.match(r"I.*", text)) #local >> see the start word

print(re.search(r"enjoying.*", text))
print(re.match(r"enjoying.*", text))
```

```
<re.Match object; span=(0, 29), match='I am enjoying the NLP course.'>
<re.Match object; span=(0, 29), match='I am enjoying the NLP course.'>
<re.Match object; span=(5, 29), match='enjoying the NLP course.'>
None
```

In [4]: `import re`

```
text = '1999 was the year I was born.'

print(re.search(r"[a-zA-Z]+", text))
print(re.match(r"[a-zA-Z]+", text))
```

```
<re.Match object; span=(5, 8), match='was'>
None
```

Search does a global search, whereas match does a local search!

Match is often used when you need to check if the string starts with a specific pattern.

Search is used when you want to find a pattern anywhere within the string.

findall

The findall function looks for all the pattern matches in the target string (whereas search and match look only for the first occurrence). The former returns a list, and the latter returns an iterator.

In [5]: `import re`

```
text1 = "a 1 b 2 c 3"
text2 = "a1 b2 c 3"
text3 = "a 1 b 222 c 3"

print(re.findall(r"\d+", text1)) #match any letter or digit
print(re.findall(r"\d+", text2))
print(re.findall(r"\d+", text3))
```

```
['1', '2', '3']
['1', '2', '3']
['1', '222', '3']
```

sub

The sub is a function that finds patterns in the target string and substitute them with

another string.

flags=re.I --> non-case-sensitive

count = specify number of matches you want to replace

```
In [6]: import re

text = 'I am enjoying the Math course.'

print(re.sub(r"Math", "NLP", text, 1, flags=re.I)) #flags=re.I >> not mater if
                                                    # 1 >> number of word i nee
```

I am enjoying the NLP course.

compile

The compile function compiles a regular expression into a regular expression object, which allows for caching and faster pattern matching.

```
In [7]: import re

text = "This year is 2021"
pattern = re.compile(r"\d+")

print(pattern.sub("2022", text))
```

This year is 2022

split

The split function is similar to the Python split function, but splits according to regular expression patterns.

```
In [8]: import re

text = "a 1 b 2 c 3"

print(re.split(r"\d+", text))
```

['a ', ' b ', ' c ', '']

Text pre-processing steps

Text preprocessing involves transforming text into a clean and consistent format that can then be fed into a model for further analysis and learning. Raw text data might contain unwanted or unimportant text due to which our results might not give efficient accuracy, and might make it hard to understand and analyze.

The various text preprocessing steps are:

1. Tokenization.
2. Lower casing.
3. Stop words removal.
4. Stemming.
5. Lemmatization.

Before implementing the pre-processing, let us understand the concept first.

Tokenization

Tokenization is used in NLP to split paragraphs and sentences into smaller units that can be more easily assigned meaning.

The first step of the NLP process is gathering the data (a sentence) and breaking it into understandable parts (words).

- **Sentence Tokenization**

```
In [9]: import nltk

text = "I'm enjoying the NLP course! I am also learning new concepts."

# print(nltk.sent_tokenize(text)) #base on . >> which is the end of sentence
```

- **Word Tokenzitaion**

```
In [10]: text = "I'm enjoying the NLP course!"
text.split()

Out[10]: ['I'm', 'enjoying', 'the', 'NLP', 'course!']
```

```
In [ ]: import nltk

text = "I'm enjoying the NLP course!"

print(nltk.word_tokenize(text))
```

```
In [ ]: !pip install -U spacy
!python -m spacy download en_core_web_sm
```

```
In [ ]: import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp("I'm enjoying the NLP course!")

for token in doc:
    print(token)
```

```
In [1]: text = "A 5km NYC cab ride costs $10.30"
doc = nlp("A 5km NYC cab ride costs $10.30")
print('nltk', nltk.word_tokenize(text))

print('#####')
for token in doc:
    print(token)
```

Lower Casting

Converting a word to lower case (NLP -> nlp). Words like Book and book mean the same but when not converted to the lower case those two are represented as two different words in the vector space model (resulting in more dimensions).

```
In [12]: text = "I'm enjoying the NLP course!"
text = text.lower()
text
```

```
Out[12]: "i'm enjoying the nlp course!"
```

Stemming

Stemming is basically removing the suffix from a word and reduce it to its root word. For example: "Flying" is a word and its suffix is "ing", if we remove "ing" from "Flying" then we will get base word or root word which is "Fly". We uses these suffix to create a new word from original stem word.

<https://www.nltk.org/howto/stem.html>

What is PorterStemmer?

It is one of the most popular stemming methods proposed in 1980. It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes. This stemmer is known for its speed and simplicity. The main applications of Porter Stemmer include data mining and Information retrieval. However, its applications are only limited to English words. Also, the group of stems is mapped on to the same stem and the output stem is not necessarily a meaningful word. The algorithms are fairly lengthy in nature and are known to be the oldest stemmer.

Advantage: It produces the best output as compared to other stemmers and it has less error rate.

Limitation: Morphological variants produced are not always real words.

```
In [13]: import nltk
from nltk.stem import PorterStemmer
```

```

ps = PorterStemmer()

words = ['run', 'runner', 'running', 'ran', 'runs', 'easily', 'fairly']

for word in words:
    print(word + ' --> ' + ps.stem(word))

```

```

run --> run
runner --> runner
running --> run
ran --> ran
runs --> run
easily --> easili
fairly --> fairli

```

What is SnowballStemmer?

It is a stemming algorithm which is also known as the Porter2 stemming algorithm as it is a better version of the Porter Stemmer since some issues of it were fixed in this stemmer.

Advantage: It is slightly faster computation time than porter, with a reasonably large community around it.

```

In [14]: import nltk
from nltk.stem.snowball import SnowballStemmer

sn = SnowballStemmer(language='english')

words = ['run', 'runner', 'running', 'ran', 'runs', 'easily', 'fairly']

for word in words:
    print(word + ' --> ' + sn.stem(word))

```

```

run --> run
runner --> runner
running --> run
ran --> ran
runs --> run
easily --> easili
fairly --> fair

```

Comparing between porter and snowball

```

In [15]: words = ['generous', 'generation', 'generously', 'generate']

for word in words:
    print(word + ' --> ' + ps.stem(word))
    print(word + ' --> ' + sn.stem(word))
    print('-----')

```

```
generous --> gener
generous --> generous
```

```
generation --> gener
generation --> generat
```

```
generously --> gener
generously --> generous
```

```
generate --> gener
generate --> generat
```

Note: Spacy does not provide stemming

Lemmatization

Lemmatization is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item. Lemmatization is similar to stemming but it brings context to the words. So it links words with similar meanings to one word.

One major difference with stemming is that lemmatize takes a part of speech parameter, "pos". If not supplied, the default is "noun."

In [16]: `%pip install nltk`

```
Requirement already satisfied: nltk in /Users/firas/miniconda3/envs/default/
lib/python3.12/site-packages (3.9.2)
Requirement already satisfied: click in /Users/firas/miniconda3/envs/default/
lib/python3.12/site-packages (from nltk) (8.4.2)
Requirement already satisfied: joblib in /Users/firas/miniconda3/envs/default/
lib/python3.12/site-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /Users/firas/miniconda3/en
vs/default/lib/python3.12/site-packages (from nltk) (2026.7.10)
Requirement already satisfied: tqdm in /Users/firas/miniconda3/envs/default/
lib/python3.12/site-packages (from nltk) (4.67.1)
Note: you may need to restart the kernel to use updated packages.
```

```
In [1]: import nltk
from nltk.stem import WordNetLemmatizer

text = 'I am enjoying with AI courses. I am taking NLP course.'
lemmatizer = WordNetLemmatizer()

for word in nltk.word_tokenize(text):
    print(f"{word}: ", lemmatizer.lemmatize(word))
```

You may go through Spacy's lemmatizer: <https://spacy.io/api/lemmatizer>

Stop Words Removal

The words which are generally filtered out before processing a natural language are

called stop words. These are actually the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc) and does not add much information to the text. Examples of a few stop words in English are "the", "a", "an", "so", "what".

Why do we need to remove stopwords?

By removing these words, we remove the low-level information from our text in order to give more focus to the important information. In other words, we can say that the removal of such words does not show any negative consequences on the model we train for our task.

- reduces the dataset size
- reduces the training time

Do we always remove stopwords? **NO!**

We do not always remove the stop words. The removal of stop words is highly dependent on the task we are performing and the goal we want to achieve. For example, if we are training a model that can perform the sentiment analysis task, we might not remove the stop words.

Movie review: "The movie was not good at all."

Text after removal of stop words: "movie good"

```
In [ ]: import spacy
        nlp = spacy.load('en_core_web_sm')
```

```
In [ ]: print(nlp.Defaults.stop_words)
```

```
In [ ]: print(nlp.vocab['myself'].is_stop)
        print(nlp.vocab['mystery'].is_stop)
```

```
In [ ]: nlp.Defaults.stop_words.add('btw')
        print (nlp.vocab['btw'].is_stop)
```

```
In [ ]: nlp.Defaults.stop_words.remove('beyond')
        nlp.vocab['beyond'].is_stop = False
```

```
In [ ]: import nltk
        nltk.download('stopwords')
```

```
In [ ]: from nltk.corpus import stopwords

        stop_words = set(stopwords.words('english'))
        print(len(stop_words))
        stop_words
```

Text pre-processing Example

To make you understand better about the need of text pre-processing. We will do a small project. In this project, I need to find how many hashtags and the top 10 hashtags used in the dataset.

The dataset is about people tweets about apple company in the twitter. You can find the dataset here: <https://www.kaggle.com/datasets/seriousran/appletwittersentimenttexts>

Output labels:

- -1: negative
 - 0: neutral
 - 1: positive
-

Task 1:

```
In [21]: import pandas as pd

!curl -L -o appletwittersentimenttexts.zip "https://www.kaggle.com/api/v1/datasets/seriousran/appletwittersentimenttexts.zip"

df = pd.read_csv('appletwittersentimenttexts.zip', encoding="utf-8", engine='python')
print(df.shape)
print(df.columns)
df.head()
```

	% Total ent	% Received	% Xferd	Average Speed Dload	Upload	Time Total	Time Spent	Time Left	Curr Speed
d	0	0	0	0	0	--:--:--	--:--:--	--:--:--	
0	100	72694	100	72694	0	0	76089	0	--:--:-- 406
2k	(1630, 2)								

```
Index(['text', 'sentiment'], dtype='object')
```

```
Out[21]:
```

	text	sentiment
0	Wow. Yall needa step it up @Apple RT @heynyla:...	-1
1	What Happened To Apple Inc? http://t.co/FJEX...	0
2	Thank u @apple I can now compile all of the pi...	1
3	The oddly uplifting story of the Apple co-foun...	0
4	@apple can i exchange my iphone for a differen...	0

```
In [22]: def preprocess(text: str) -> str:
          text = str(text).lower()
          text = re.sub(r"http\S+|www.\S+", " ", text)
          text = re.sub(r"@w+", " ", text)
          text = re.sub(r"^[a-z0-9#\s]", " ", text) # keep letters and digits
          text = re.sub(r"\s+", " ", text).strip() # fix whitespace
          return text
```

```
df["clean_text"] = df["text"].astype(str).apply(preprocess)
```

```
In [23]: hashtag_pattern = re.compile(r"#\w+") # '#' followed by atleast 1 word
```

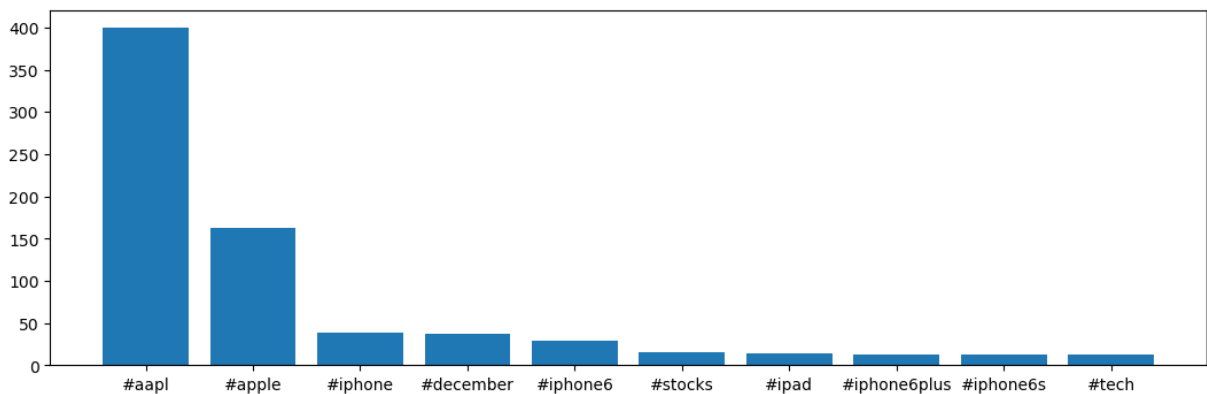
```
all_hashtags = []
for t in df["text"].fillna("").astype(str).str.lower():
    all_hashtags.extend(hashtag_pattern.findall(t))
```

```
In [24]: accum = dict()
          for key in all_hashtags:
              accum[key] = accum.get(key, 0) + 1
```

```
In [25]: top10 = sorted(accum.items(), reverse=True, key=lambda x: x[1])[:10]
```

```
In [26]: xs = [x[0] for x in top10]
          ys = [y[1] for y in top10]
```

```
In [27]: import matplotlib.pyplot as plt
          plt.figure(figsize=(13,4))
          plt.bar(x=xs, height=ys);
```



Lab 2 Complete.